

TEK BANTLI TURING MAKİNESİ İLE BINARY ÇARPMA HESAPLAYICI

Kısa Proje Raporu

Proje Konusu: Tek Bantlı Turing Makinesi ile İkili Sayı Çarpma Makinesi

Hazırlayan: Furkan Elidolu

Programlama Dili: Python

Bu rapor; problem tanımı, Turing Makinesi modeli, binary sayı sistemi, operand ayrıştırma yaklaşımı, durumların açıklaması ve geçiş mantığı bölümlerini içerir.

Rapor Kapsamı

Bu çalışmada Python programlama dili kullanılarak tek bantlı bir Turing Makinesi simülatörü tasarlanmıştır. Program, kullanıcıdan aldığı iki binary sayıyı bant üzerinde `birinci_sayı*ikinci_sayı=` formatında temsil eder, * sembolünü kullanarak operandları ayırır ve çarpma işlemi kaydır ve topla yaklaşımıyla gerçekleştirir.

Raporda özellikle ödevin zorunlu tasarım gereksinimi olan operand ayrıştırma yaklaşımı ayrıntılı biçimde açıklanmıştır. Ayrıca makinenin durumları, bant yapısı, okuma/yazma kafası, geçiş mantığı ve programdaki karşılıkları sistemli şekilde sunulmuştur.

1. Problem Tanımı

Bu projenin amacı, binary tabanda verilen iki sayının çarpımını tek bantlı Turing Makinesi mantığıyla modelleyen bir Python programı geliştirmektir. Kullanıcı programı çalıştırdığında iki adet binary sayı girer. Program önce bu girişlerin yalnızca 0 ve 1 karakterlerinden oluşup oluşmadığını kontrol eder. Geçersiz bir karakter bulunursa makine işlemi reddeder ve kullanıcıya hata mesajı verir.

Geçerli girişler alındıktan sonra iki sayı bant üzerinde özel bir formatla temsil edilir. Birinci sayı ile ikinci sayı arasına * ayırıcı konur ve sonucun yazılacağı alan = sembolüyle belirtilir. Örneğin kullanıcı birinci sayı olarak 11, ikinci sayı olarak 10 girerse programın başlangıç bant içeriği aşağıdaki gibi olur:

11*10=

Bu gösterimde * sembolünün sol tarafı birinci operandı, sağ tarafı ikinci operandı temsil eder. = sembolünden sonraki alan ise çarpma sonucunun yazılacağı bölgedir. Böylece bant üzerinde hem veriler hem de sonuç alanı açık biçimde ayrılmış olur.

Programın temel problemi, bu bant yapısı üzerinde önce operandları doğru şekilde ayırmak, ardından ikinci sayının bitlerini sağdan sola okuyarak kaydır ve topla mantığı ile çarpma işlemi gerçekleştirmektir. Sonuç hem binary olarak hem de decimal karşılığıyla kullanıcıya gösterilir.

Gereksinim	Programdaki Karşılığı
Kullanıcıdan iki binary sayı alma	<code>main()</code> fonksiyonu içinde <code>input()</code> ile birinci ve ikinci sayı alınır.
Binary doğrulama	<code>is_binary()</code> fonksiyonu boş girişleri ve 0/1 dışındaki karakterleri reddeder.
Bant formatı oluşturma	<code>TuringMachine.__init__</code> içinde <code>first_number + "*" + second_number + "="</code> yapısı listeye çevrilir.
Operand ayrıştırma	<code>find_star()</code> , <code>find_equal()</code> ve <code>split_operands()</code> fonksiyonlarıyla * ve = konumları bulunur.
Çarpma işlemi	<code>multiply_with_shift_add()</code> fonksiyonu multiplier bitlerini sağdan sola okuyarak kaydır-topla işlemi yürütür.
Sonucun yazılması	<code>write_result_to_tape()</code> fonksiyonu sonucu = sembolünden sonra banda yazar.
Kabul / red durumu	<code>accept()</code> başarılı işlem sonunda <code>q_accept</code> durumunu; <code>reject()</code> hata durumunda <code>q_reject</code> durumunu gösterir.

2. Turing Makinesi Modeli

Turing Makinesi, sınırsız kabul edilen bir bant, bu bant üzerinde hareket eden bir okuma/yazma kafası, durumlar kümesi ve geçiş kurallarından oluşan soyut bir hesaplama modelidir. Bu projede geliştirilen program da aynı temel bileşenleri Python sınıfı içinde temsil eder.

Modelde bant, Python listesi olarak tutulmuştur. Böylece bant üzerindeki her sembol ayrı bir hücre gibi düşünülebilir. Okuma/yazma kafası ise head değişkeniyle temsil edilir. head değişkeni hangi bant hücresinin okunacağını ve gerektiğinde hangi hücreye yazılacağını belirler.

Makinenin biçimsel gösterimi aşağıdaki bileşenlerle açıklanabilir:

Bileşen	Tanım	Bu Projedeki Karşılığı
Q	Durumlar kümesi	q_start, q_find_star, q_find_equal, q_split_operands, q_read_multiplier_bit, q_skip_add, q_shift_multiplicand, q_add, q_write_result, q_accept, q_reject
Σ	Giriş alfabesi	{0, 1, *, =}
Γ	Bant alfabesi	{0, 1, *, =, _}
δ	Geçiş fonksiyonu	Okunan sembole ve mevcut duruma göre yeni durum, yazılacak sembol ve kafa hareketi belirlenir.
q0	Başlangıç durumu	q_start
B	Boşluk sembolü	
F	Kabul durumu	q_accept
Red durumu	Hatalı durum	q_reject

Programda read_symbol() fonksiyonu kafanın bulunduğu hücredeki sembolü okur. write_symbol() fonksiyonu ilgili hücreye yeni sembol yazar. move_head() fonksiyonu ise kafayı sağa, sola veya aynı konumda kalacak şekilde hareket ettirir. Böylece Turing Makinesinin temel okuma, yazma ve hareket etme davranışları doğrudan simüle edilmiş olur.

Her geçiş sırasında print_step() fonksiyonu mevcut durumu, okunan sembolü, yazılan sembolü, kafa hareketini ve bant içeriğini ekrana yazdırır. Bu sayede program yalnızca sonucu hesaplamakla kalmaz, aynı zamanda Turing Makinesi simülasyonunun adım adım izlenmesini sağlar.

3. Binary Sayı Sistemi

Binary sayı sistemi yalnızca iki sembolden oluşur: 0 ve 1. Bilgisayar sistemlerinde veriler ve aritmetik işlemler temelde binary gösterim üzerinden işlenir. Decimal sistemde basamak değerleri 10'un kuvvetlerine göre belirlenirken binary sistemde basamak değerleri 2'nin kuvvetlerine göre belirlenir.

Örneğin 11_2 sayısı decimal olarak 3 değerine eşittir. Çünkü soldaki 1 değeri 2^1 basamağını, sağdaki 1 değeri ise 2^0 basamağını temsil eder:

$$11_2 = 1 \times 2^1 + 1 \times 2^0 = 2 + 1 = 3$$

Benzer şekilde 10_2 sayısı decimal olarak 2 değerine eşittir:

$$10_2 = 1 \times 2^1 + 0 \times 2^0 = 2 + 0 = 2$$

Bu iki sayı çarpıldığında decimal karşılığı $3 \times 2 = 6$ olur. 6 sayısının binary karşılığı ise 110_2 şeklindedir. Dolayısıyla örnek işlemde beklenen çıktı aşağıdaki gibidir:

$$11_2 \times 10_2 = 110_2$$

Bu projede çarpma işlemi doğrudan hazır bir çarpma operatörüyle yapılmamıştır. Bunun yerine bilgisayar mimarisinde de kullanılan kaydır ve topla yaklaşımı temel alınmıştır. Bu yöntemde ikinci sayının bitleri sağdan sola okunur. Okunan bit 1 ise birinci sayı ilgili basamak kadar sola kaydırılarak ara toplama eklenir. Okunan bit 0 ise o basamak için toplama yapılmaz.

Multiplier biti	Yapılan işlem	Anlamı
0	Toplama yapılmaz.	Bu basamağın çarpıma katkısı yoktur.
1	Multiplicand sola kaydırılarak sonuca eklenir.	Bu basamağın çarpıma katkısı vardır.

4. Operand Ayırıştırma Yaklaşımı

Bu ödevin en kritik bölümü operandların bant üzerinde doğru ayrıştırılmasıdır. Çünkü çarpma işlemi başlamadan önce makinenin hangi sembollerin birinci sayıya, hangi sembollerin ikinci sayıya ait olduğunu kesin olarak belirlemesi gerekir. Bu projede operand ayrıştırma işlemi * ve = sembolleri kullanılarak yapılmıştır.

Bant formatı şu şekildedir:

```
birinci_sayı*ikinci_sayı=
```

Bu yapıda * sembolünün sol tarafında kalan kısım birinci sayı yani multiplicand olarak kabul edilir. * sembolünün sağında ve = sembolünün solunda kalan kısım ise ikinci sayı yani multiplier olarak kabul edilir. = sembolü ise sonuç alanının başlangıcını belirtir.

Bant Parçası	Görevi	Örnek Üzerindeki Değeri
* sembolünün sol tarafı	Birinci operand / multiplicand	11
* ile = arasındaki bölüm	İkinci operand / multiplier	10
= sembolünden sonrası	Sonucun yazılacağı alan	Başlangıçta boş, işlem sonunda 110

Program bu ayrımı üç temel adımda yapar. İlk olarak find_star() fonksiyonu bant üzerinde soldan sağa ilerleyerek * sembolünü arar. Bu sembol bulunduğunda makine, birinci operandın bittiğini ve ikinci operandın başladığını anlar. İkinci olarak find_equal() fonksiyonu = sembolünü bulur. Bu sembol ikinci operandın bittiğini ve sonuç alanının başladığını gösterir. Üçüncü olarak split_operands() fonksiyonu, bulunan indekslere göre sol ve sağ parçaları ayrı değişkenlere aktarır.

```
multiplicand = tape[0 : star_index]
multiplier   = tape[star_index + 1 : equal_index]
```

Örneğin bant içeriği 11*10= olduğunda * sembolü 2. indekste, = sembolü ise 5. indekste bulunur. Buna göre 0 ile 2 arasındaki semboller birinci sayı, 3 ile 5 arasındaki semboller ikinci sayı olarak ayrıştırılır.

İndeks	Sembol	Yorum
0	1	Birinci operandın 1. biti
1	1	Birinci operandın 2. biti
2	*	Operand ayırıcı
3	1	İkinci operandın 1. biti
4	0	İkinci operandın 2. biti
5	=	Sonuç alanı başlangıcı

Programda operandlardan biri boş bırakılırsa işlem geçersiz sayılır ve makine q_reject durumuna geçer. Böylece örneğin *10= veya 11*= gibi eksik operand içeren bant yapıları kabul edilmez. Bu kontrol, çarpma işleminin anlamlı ve doğru şekilde yapılabilmesi için gereklidir.

Operand ayrıştırma tamamlandıktan sonra multiplicand ve multiplier değerleri işlem boyunca korunur. Çarpma aşamasında multiplier sağdan sola okunur; multiplicand ise okunan bitin konumuna göre sola kaydırılarak sonuca eklenir. Bu nedenle * sembolü yalnızca görsel bir ayraç değildir; aynı zamanda algoritmanın hangi kısmı birinci sayı, hangi kısmı ikinci sayı olarak kullanacağını belirleyen temel tasarım unsurudur.

5. Durumların Açıklaması

Makinenin işlem akışı durumlar üzerinden modellenmiştir. Her durum belirli bir görevi temsil eder. Programda kullanılan durumlar ve görevleri aşağıdaki tabloda açıklanmıştır.

Durum	Görevi	Kodda İlgili Bölüm
q_start	Başlangıç durumudur. Makine çalışmaya bu durumdan başlar ve ilk sembole göre q_find_star durumuna geçer.	__init__ / run
q_find_star	Bant üzerinde * ayırıcısını arar. 0 veya 1 okudukça sağa ilerler. * bulunduğu birinci operandın bittiği anlaşılır.	find_star
q_find_equal	Bant üzerinde = sembolünü arar. Bu sembol ikinci operandın sonunu ve sonuç alanının başlangıcını gösterir.	find_equal
q_split_operands	* ve = konumlarına göre birinci ve ikinci operandı ayırır. Boş operand varsa q_reject durumuna geçilir.	split_operands
q_read_multiplier_bit	Multiplier değerinin bitlerini sağdan sola okur. Okunan bit 0 ise q_skip_add, 1 ise q_shift_multiplicand durumuna geçilir.	multiply_with_shift_add
q_skip_add	Okunan multiplier biti 0 olduğunda toplama yapılmadan bir sonraki bite geçilir.	multiply_with_shift_add
q_shift_multiplicand	Okunan multiplier biti 1 olduğunda multiplicand, bitin konumuna göre sola kaydırılır.	multiply_with_shift_add
q_add	Kaydırılmış multiplicand değeri mevcut ara sonuca binary_add() fonksiyonu ile eklenir.	multiply_with_shift_add / binary_add
q_write_result	Hesaplanan binary sonuç = sembolünden sonra banda yazılır.	write_result_to_tape
q_accept	İşlem başarıyla tamamlandığında geçilen kabul durumudur.	accept
q_reject	Geçersiz giriş, beklenmeyen sembol veya boş operand durumlarında geçilen red durumudur.	reject

Bu durum yapısı sayesinde program karmaşık bir çarpma işlemini daha küçük ve takip edilebilir aşamalara ayırır. Önce bant üzerinde ayraçlar bulunur, sonra operandlar ayrıştırılır, ardından multiplier bitleri tek tek işlenir ve son olarak sonuç banda yazılır.

6. Geçiş Mantığı

Turing Makinesinde her geçiş, mevcut durum ve okunan sembole göre belirlenir. Genel geçiş biçimi aşağıdaki gibidir:

$\delta(\text{mevcut_durum}, \text{okunan_sembol}) = (\text{yeni_durum}, \text{yazılan_sembol}, \text{kafa_hareketi})$

Bu projede kafa hareketleri üç farklı şekilde tanımlanmıştır: R sağa hareketi, L sola hareketi, S ise aynı konumda kalmayı ifade eder. Programda bu hareketler RIGHT, LEFT ve STAY sabitleriyle gösterilmiştir.

Programın geçiş mantığı iki seviyede ele alınabilir. İlk seviye, bant üzerinde * ve = sembollerini arayan klasik Turing Makinesi geçişleridir. Bu kısımda makine 0 veya 1 okuduğunda aynı sembolü yazar ve sağa ilerler. * bulunduğu birinci operand tamamlanmış olur. = bulunduğu ikinci operand tamamlanmış olur. İkinci seviye ise çarpma işlemidir. Bu aşamada multiplier bitleri sağdan sola okunur ve her bite göre farklı işlem uygulanır.

Multiplier biti 0 ise bu basamak çarpıma katkı sağlamadığı için toplama yapılmaz ve makine q_skip_add durumuna geçer. Multiplier biti 1 ise multiplicand, bitin bulunduğu basamak değerine göre sola kaydırılır ve q_add durumunda mevcut sonuca eklenir. Bütün multiplier bitleri işlendiğinde makine q_write_result durumuna geçerek sonucu = sembolünden sonra banda yazar.

6.1. Temel Geçiş Tablosu

Mevcut Durum	Okunan	Yeni Durum	Yazılan	Hareket	Açıklama
q_start	0	q_find_star	0	R	Girişin ilk biti okunur ve * arama aşamasına geçilir.
q_start	1	q_find_star	1	R	Girişin ilk biti okunur ve * arama aşamasına geçilir.
q_find_star	0	q_find_star	0	R	Birinci operand üzerinde ilerlenir.
q_find_star	1	q_find_star	1	R	Birinci operand üzerinde ilerlenir.

Mevcut Durum	Okunan	Yeni Durum	Yazılan	Hareket	Açıklama
q_find_star	*	q_find_equal	*	R	* bulunduğu için ikinci operand alanına geçilir.
q_find_equal	0	q_find_equal	0	R	İkinci operand üzerinde ilerlenir.
q_find_equal	1	q_find_equal	1	R	İkinci operand üzerinde ilerlenir.
q_find_equal	=	q_split_operands	=	S	= bulunduğu için operand ayrıştırma yapılır.
q_split_operands	*	q_read_multiplier_bit	*	S	Sol taraf multiplicand, sağ taraf multiplier olarak ayrılır.
q_read_multiplier_bit	0	q_skip_add	0	L	Multiplier biti 0 olduğu için toplama yapılmaz.
q_read_multiplier_bit	1	q_shift_multiplicand	1	S	Multiplier biti 1 olduğu için multiplicand kaydırılır.
q_skip_add	0/1	q_read_multiplier_bit	aynı	L	Bir sonraki multiplier bitine geçilir.
q_shift_multiplicand	1	q_add	1	S	Kaydırılmış değer toplama aşamasına gönderilir.
q_add	0/1	q_read_multiplier_bit	aynı	L	Bitler bitmediyse sonraki multiplier biti okunur.
q_add	tüm bitler bitti	q_write_result	aynı	S	Çarpma tamamlanır ve sonuç yazma aşamasına geçilir.
q_write_result	_	q_write_result	sonuç biti	R	Sonuç bitleri = sembolünden sonra banda yazılır.
q_write_result	sonuç bitti	q_accept	aynı	S	Makine kabul durumuna geçer.
herhangi	geçersiz sembol	q_reject	aynı	S	Hatalı giriş veya beklenmeyen sembol durumunda işlem reddedilir.

6.2. 11 × 10 Örneği Üzerinden Geçiş Akışı

Örnek olarak 11 ve 10 sayıları verildiğinde başlangıç bant içeriği 11*10= olur. Makine önce q_find_star durumunda * sembolünü arar. * bulunduğu * sembolünün sol tarafındaki 11 değeri multiplicand olarak belirlenir. Ardından q_find_equal durumunda = sembolü aranır ve * ile = arasındaki 10 değeri multiplier olarak ayrılır.

Multiplier değeri sağdan sola okunur. 10 sayısının en sağ biti 0 olduğu için ilk aşamada toplama yapılmaz. Bir sonraki bit 1 olduğu için multiplicand olan 11, bir basamak sola kaydırılır ve 110 haline gelir. Ara sonuç 0 iken 110 değeri eklendiğinde sonuç 110 olur. Son olarak q_write_result durumunda bu sonuç = sembolünden sonra banda yazılır ve final bant içeriği 11*10=110 şeklinde oluşur.

```

Başlangıç: 11*10=
Operand ayrımı: multiplicand = 11, multiplier = 10
Sağdan ilk bit: 0 -> toplama yok
Sonraki bit: 1 -> 11 sola kaydırılır -> 110
Final bant: 11*10=110

```

6.3. Geçiş Mantığının Kodla İlişkisi

Kodda temel geçişler transition_function sözlüğü içinde gösterilmiştir. Bunun yanında find_star(), find_equal(), split_operands(), multiply_with_shift_add() ve write_result_to_tape() fonksiyonları geçişlerin çalışma davranışını adım adım uygular. apply_transition() fonksiyonu ise bir geçiş sırasında okuma, yazma, adım çıktısı üretme, kafa hareketi ve yeni duruma geçme işlemlerini birlikte yürütür.

Bu yapı sayesinde program hem teorik Turing Makinesi modelini hem de pratik binary çarpma algoritmasını bir arada gösterir. Her kritik aşama ayrı bir durumla ifade edildiği için çalışma süreci takip edilebilir, ekran çıktılarında mevcut durum ve bant içeriği açık şekilde görülebilir.

7. Girdi-Çıktı Örnekleri

Bu bölümde geliştirilen Turing Makinesi simülatörü farklı binary sayı çiftleriyle test edilmiştir. Testlerde amaç, programın hem küçük değerlerde hem de birden fazla bitten oluşan binary sayılarda doğru sonuç üretip üretmediğini kontrol etmektir. Her testte kullanıcıdan iki binary sayı alınmış, bu sayılar $\text{birinci_sayı} * \text{ikinci_sayı} =$ bant formatına dönüştürülmüş ve çarpma işlemi kaydır ve topla mantığıyla gerçekleştirilmiştir.

Program çıktısında final bant içeriği, binary sonuç ve sonucun decimal karşılığı gösterilmektedir. Böylece hem Turing Makinesi bandındaki sonuç hem de matematiksel doğruluk kontrol edilebilmektedir.

Test No	Birinci Binary Sayı	İkinci Binary Sayı	Bant Formatı	Beklenen Binary Sonuç	Decimal Karşılığı
1	0	0	$0*0=$	0	0
2	1	0	$1*0=$	0	0
3	1	1	$1*1=$	1	1
4	11	10	$11*10=$	110	6
5	101	11	$101*11=$	1111	15
6	111	101	$111*101=$	100011	35

Test 1 Açıklaması

Birinci testte 0 ve 0 sayıları çarpılmıştır. Binary sistemde $0 \times 0 = 0$ olduğu için programın üretmesi gereken sonuç 0 olmalıdır. Program final bant içeriğinde $0*0=0$ sonucunu üretir. Bu test, sıfır değerinin doğru işlendiğini gösterir.

Test 2 Açıklaması

İkinci testte 1 ve 0 sayıları çarpılmıştır. Bir sayının sıfır ile çarpımı her zaman sıfırdır. Bu nedenle beklenen sonuç 0 değeridir. Program $1*0=0$ sonucunu vererek çarpanın sıfır olduğu durumda toplama işlemi yapılmadan doğru sonuca ulaşmaktadır.

Test 3 Açıklaması

Üçüncü testte 1 ve 1 sayıları çarpılmıştır. Binary sistemde $1 \times 1 = 1$ olduğu için beklenen sonuç 1 değeridir. Bu test, en küçük pozitif binary değerlerin çarpımında programın doğru çalıştığını göstermektedir.

Test 4 Açıklaması

Dördüncü testte 11 ve 10 sayıları çarpılmıştır. 11_2 decimal sistemde 3'e, 10_2 decimal sistemde 2'ye karşılık gelir. Bu nedenle sonuç decimal olarak 6, binary olarak ise 110 olmalıdır. Program final bant içeriğini $11*10=110$ şeklinde üretir. Bu test, ödevde verilen örnek ile uyumlu olduğu için temel doğrulama testi olarak kullanılmıştır.

Test 5 Açıklaması

Beşinci testte 101 ve 11 sayıları çarpılmıştır. 101_2 decimal olarak 5, 11_2 decimal olarak 3 değerindedir. $5 \times 3 = 15$ olduğu için beklenen binary sonuç 1111 değeridir. Program bu sonucu üretirken birden fazla toplama ve kaydırma adımının doğru çalıştığını göstermektedir.

Test 6 Açıklaması

Altıncı testte 111 ve 101 sayıları çarpılmıştır. 111_2 decimal olarak 7, 101_2 decimal olarak 5 değerindedir. $7 \times 5 = 35$ olduğundan beklenen binary sonuç 100011 değeridir. Bu test, daha uzun bitli sayıların çarpımında da programın doğru sonuç verdiğini göstermektedir.

Bu testler sonucunda programın farklı binary girişlerde doğru şekilde çalıştığı görülmüştür. Özellikle * sembolü ile operandların ayrılması, multiplier değerinin sağdan sola okunması, bit değerine göre toplama yapılması veya atlanması ve sonucun = sembolünden sonra banda yazılması başarılı şekilde gerçekleştirilmiştir.

Program Ekran Görüntüleri

Aşağıda 2 örnek bulunuyor gönderilen zip dosyasında Test Örneklerinde de 6 örneğin adım adım ekran görüntüsü mevcut

```
Birinci binary sayıyı giriniz: 11
İkinci binary sayıyı giriniz: 10

Başlangıç bant içeriği: 11*10=
Bant formatı: birinci_sayı*ikinci_sayı=
```

```
-----
Adım: 1
Mevcut durum: q_find_star
Okunan sembol: 1
Yazılan sembol: 1
Kafa hareketi: R
Bant içeriği: 11*10=
                ^
```

```
-----
Adım: 2
Mevcut durum: q_find_star
Okunan sembol: 1
Yazılan sembol: 1
Kafa hareketi: R
Bant içeriği: 11*10=
                ^
```

```
-----
Adım: 3
Mevcut durum: q_find_star
Okunan sembol: *
Yazılan sembol: *
Kafa hareketi: R
Bant içeriği: 11*10=
                ^
```

```
-----
Adım: 4
Mevcut durum: q_find_equal
Okunan sembol: 1
Yazılan sembol: 1
Kafa hareketi: R
Bant içeriği: 11*10=
                ^
```

```
-----
Adım: 5
Mevcut durum: q_find_equal
Okunan sembol: 0
Yazılan sembol: 0
Kafa hareketi: R
Bant içeriği: 11*10=
                ^
```

```
-----
Adım: 6
Mevcut durum: q_find_equal
Okunan sembol: =
Yazılan sembol: =
Kafa hareketi: S
Bant içeriği: 11*10=
                ^
```

```
-----
Adım: 7
Mevcut durum: q_split_operands
Okunan sembol: *
Yazılan sembol: *
Kafa hareketi: S
Bant içeriği: 11*10=
               ^

Operand ayrıştırma:
* sol tarafı, birinci sayı: 11
* sağ tarafı, ikinci sayı: 10

-----

Adım: 8
Mevcut durum: q_read_multiplier_bit
Okunan sembol: 0
Yazılan sembol: 0
Kafa hareketi: S
Bant içeriği: 11*10=
               ^

-----

Adım: 9
Mevcut durum: q_skip_add
Okunan sembol: 0
Yazılan sembol: 0
Kafa hareketi: L
Bant içeriği: 11*10=
               ^

-----

Adım: 10
Mevcut durum: q_read_multiplier_bit
Okunan sembol: 1
Yazılan sembol: 1
Kafa hareketi: S
Bant içeriği: 11*10=
               ^

-----

Adım: 11
Mevcut durum: q_shift_multiplicand
Okunan sembol: 1
Yazılan sembol: 1
Kafa hareketi: S
Bant içeriği: 11*10=
               ^

-----

Adım: 12
Mevcut durum: q_add
Okunan sembol: 1
Yazılan sembol: 1
Kafa hareketi: S
Bant içeriği: 11*10=
               ^
Ara toplam: 0 + 110 = 110
```

```
-----  
Adım: 12  
Mevcut durum: q_add  
Okunan sembol: 1  
Yazılan sembol: 1  
Kafa hareketi: S  
Bant içeriği: 11*10=  
                ^
```

Ara toplam: $0 + 110 = 110$

```
-----  
Adım: 13  
Mevcut durum: q_write_result  
Okunan sembol: _  
Yazılan sembol: 1  
Kafa hareketi: R  
Bant içeriği: 11*10=1  
                ^
```

```
-----  
Adım: 14  
Mevcut durum: q_write_result  
Okunan sembol: _  
Yazılan sembol: 1  
Kafa hareketi: R  
Bant içeriği: 11*10=11  
                ^
```

```
-----  
Adım: 15  
Mevcut durum: q_write_result  
Okunan sembol: _  
Yazılan sembol: 0  
Kafa hareketi: R  
Bant içeriği: 11*10=110  
                ^
```

```
-----  
Adım: 16  
Mevcut durum: q_accept  
Okunan sembol: 0  
Yazılan sembol: 0  
Kafa hareketi: S  
Bant içeriği: 11*10=110  
                ^
```

=====

SONUÇ

=====

Final bant içeriği: 11*10=110
Binary sonuç: 110
Decimal karşılığı: 6

=====

furkanelidolu@Furkan-MacBook-Pro ~ %

```
furkanelidolu@Furkan-MacBook-Pro ~ % /opt/homebrew/bin/python3 "/Users/furkanelidolu/Desktop/ODEV/Ödev 1/main_odev1.py"
Turing Makinesi ile Binary Çarpma Hesaplayıcı
-----
Birinci binary sayıyı giriniz: 101
İkinci binary sayıyı giriniz: 110

Başlangıç bant içeriği: 101*110=
Bant formatı: birinci_sayı*ikinci_sayı=

-----
Adım: 1
Mevcut durum: q_find_star
Okunan sembol: 1
Yazılan sembol: 1
Kafa hareketi: R
Bant içeriği: 101*110=
      ^
-----
Adım: 2
Mevcut durum: q_find_star
Okunan sembol: 0
Yazılan sembol: 0
Kafa hareketi: R
Bant içeriği: 101*110=
      ^
-----
Adım: 3
Mevcut durum: q_find_star
Okunan sembol: 1
Yazılan sembol: 1
Kafa hareketi: R
Bant içeriği: 101*110=
      ^
-----
```

```
-----  
Adım: 4  
Mevcut durum: q_find_star  
Okunan sembol: *  
Yazılan sembol: *  
Kafa hareketi: R  
Bant içeriği: 101*110=  
                ^  
  
-----  
Adım: 5  
Mevcut durum: q_find_equal  
Okunan sembol: 1  
Yazılan sembol: 1  
Kafa hareketi: R  
Bant içeriği: 101*110=  
                ^  
  
-----  
Adım: 6  
Mevcut durum: q_find_equal  
Okunan sembol: 1  
Yazılan sembol: 1  
Kafa hareketi: R  
Bant içeriği: 101*110=  
                ^  
  
-----  
Adım: 7  
Mevcut durum: q_find_equal  
Okunan sembol: 0  
Yazılan sembol: 0  
Kafa hareketi: R  
Bant içeriği: 101*110=  
                ^  
  
-----  
Adım: 8  
Mevcut durum: q_find_equal  
Okunan sembol: =  
Yazılan sembol: =  
Kafa hareketi: S  
Bant içeriği: 101*110=  
                ^  
  
-----  
Adım: 9  
Mevcut durum: q_split_operands  
Okunan sembol: *  
Yazılan sembol: *  
Kafa hareketi: S  
Bant içeriği: 101*110=  
                ^  
  
Operand ayrıştırma:  
* sol tarafı, birinci sayı: 101  
* sağ tarafı, ikinci sayı: 110
```

```
-----  
Adım: 10  
Mevcut durum: q_read_multiplier_bit  
Okunan sembol: 0  
Yazılan sembol: 0  
Kafa hareketi: S  
Bant içeriği: 101*110=  
                ^
```

```
-----  
Adım: 11  
Mevcut durum: q_skip_add  
Okunan sembol: 0  
Yazılan sembol: 0  
Kafa hareketi: L  
Bant içeriği: 101*110=  
                ^
```

```
-----  
Adım: 12  
Mevcut durum: q_read_multiplier_bit  
Okunan sembol: 1  
Yazılan sembol: 1  
Kafa hareketi: S  
Bant içeriği: 101*110=  
                ^
```

```
-----  
Adım: 13  
Mevcut durum: q_shift_multiplicand  
Okunan sembol: 1  
Yazılan sembol: 1  
Kafa hareketi: S  
Bant içeriği: 101*110=  
                ^
```

```
-----  
Adım: 14  
Mevcut durum: q_add  
Okunan sembol: 1  
Yazılan sembol: 1  
Kafa hareketi: S  
Bant içeriği: 101*110=  
                ^  
Ara toplam: 0 + 1010 = 1010
```

```
-----  
Adım: 15  
Mevcut durum: q_read_multiplier_bit  
Okunan sembol: 1  
Yazılan sembol: 1  
Kafa hareketi: S  
Bant içeriği: 101*110=  
                ^
```

```
-----  
Adım: 16  
Mevcut durum: q_shift_multiplicand  
Okunan sembol: 1  
Yazılan sembol: 1  
Kafa hareketi: S  
Bant içeriği: 101*110=  
                ^
```

```
-----  
Adım: 17  
Mevcut durum: q_add  
Okunan sembol: 1  
Yazılan sembol: 1  
Kafa hareketi: S  
Bant içeriği: 101*110=  
                ^  
Ara toplam: 1010 + 10100 = 11110
```

```
-----  
Adım: 18  
Mevcut durum: q_write_result  
Okunan sembol: _  
Yazılan sembol: 1  
Kafa hareketi: R  
Bant içeriği: 101*110=1  
                ^
```

```
-----  
Adım: 19  
Mevcut durum: q_write_result  
Okunan sembol: _  
Yazılan sembol: 1  
Kafa hareketi: R  
Bant içeriği: 101*110=11  
                ^
```

```
-----  
Adım: 20  
Mevcut durum: q_write_result  
Okunan sembol: _  
Yazılan sembol: 1  
Kafa hareketi: R  
Bant içeriği: 101*110=111  
                ^
```

```
-----  
Adım: 21  
Mevcut durum: q_write_result  
Okunan sembol: _  
Yazılan sembol: 1  
Kafa hareketi: R  
Bant içeriği: 101*110=1111  
                ^
```

```
-----  
Adım: 22  
Mevcut durum: q_write_result  
Okunan sembol: _  
Yazılan sembol: 0  
Kafa hareketi: R  
Bant içeriği: 101*110=11110  
                ^
```

```
-----  
Adım: 23  
Mevcut durum: q_accept  
Okunan sembol: 0  
Yazılan sembol: 0  
Kafa hareketi: S  
Bant içeriği: 101*110=11110  
                ^
```



```

=====
Adım: 23
Mevcut durum: q_accept
Okunan sembol: 0
Yazılan sembol: 0
Kafa hareketi: S
Bant içeriği: 101*110=11110
               ^

```

SONUÇ

```

=====
Final bant içeriği: 101*110=11110
Binary sonuç: 11110
Decimal karşılığı: 30
=====

```

furkanelidolu@Furkan-MacBook-Pro: ~ %

9. Sonuç ve Değerlendirme

Bu projede, Python programlama dili kullanılarak tek bantlı bir Turing Makinesi simülatörü geliştirilmiştir. Geliştirilen makine, kullanıcıdan aldığı iki binary sayıyı * ve = sembolleriyle bant formatına dönüştürmekte ve çarpma işlemini Turing Makinesi mantığına uygun şekilde adım adım gerçekleştirmektedir.

Programın en önemli bölümlerinden biri operand ayrıştırma mekanizmasıdır. Bu mekanizmada * sembolünün sol tarafı birinci sayı yani multiplicand, sağ tarafı ise ikinci sayı yani multiplier olarak belirlenmiştir. Makine önce bant üzerinde * sembolünü bulmakta, ardından = sembolünü bularak ikinci operandın sınırını belirlemektedir. Bu sayede çarpma işlemine geçmeden önce iki sayı bant üzerinde doğru şekilde ayrıştırılmaktadır.

Çarpma işlemi, binary sistemde yaygın olarak kullanılan kaydır ve topla yöntemiyle gerçekleştirilmiştir. Bu yöntemde multiplier değeri sağdan sola doğru okunur. Okunan bit 0 ise toplama işlemi yapılmaz. Okunan bit 1 ise multiplicand değeri ilgili basamak kadar sola kaydırılır ve ara sonuca eklenir. Bu işlem tüm multiplier bitleri bitene kadar devam eder. Son olarak elde edilen binary sonuç = sembolünden sonra banda yazılır.

Program, her adımda mevcut durum, okunan sembol, yazılan sembol, kafa hareketi ve bant içeriğini ekrana yazdırmaktadır. Bu özellik sayesinde Turing Makinesi'nin çalışma süreci yalnızca sonuç odaklı değil, adım adım izlenebilir şekilde tasarlanmıştır. Böylece makinenin hangi durumda hangi sembolü okuduğu, hangi sembolü yazdığı ve kafanın hangi yöne hareket ettiği açıkça görülebilmektedir.

Yapılan testler sonucunda programın farklı binary sayı çiftleri için doğru sonuçlar ürettiği görülmüştür. $0*0$, $1*0$, $1*1$, $11*10$, $101*11$ ve $111*101$ gibi farklı testlerde program hem binary sonucu hem de decimal karşılığını doğru şekilde hesaplamıştır. Bu testler, programın hem basit durumlarda hem de daha uzun bitli sayılarda doğru çalıştığını göstermektedir.

Sonuç olarak bu proje ile Turing Makinesi'nin temel bileşenleri olan bant, okuma/yazma kafası, durum kümesi, giriş alfabesi, bant alfabesi, geçiş mantığı, başlangıç durumu, kabul durumu ve red durumu Python üzerinde modellenmiştir. Ayrıca binary çarpma işleminin yalnızca matematiksel bir işlem olarak değil, durum tabanlı ve adım adım ilerleyen bir hesaplama süreci olarak nasıl gerçekleştirilebileceği gösterilmiştir.

Bu çalışma, Turing Makinesi modelinin teorik yapısını pratik bir programlama uygulamasıyla birleştirmiştir. Proje sonucunda binary sayıların çarpılması, operandların bant üzerinde ayrıştırılması, durum geçişlerinin uygulanması ve sonucun banda yazılması başarılı şekilde gerçekleştirilmiştir. Bu nedenle geliştirilen program, ödev kapsamında istenen Turing Makinesi ile binary çarpma hesaplayıcı amacını karşılamaktadır.